

RELATORIO PRATICO

Cloud DevOps: Orchestrating Containers and Microservices

1. Ambiente Local com Docker Compose

1.1 Arquitetura Multi-Servico

O ambiente de desenvolvimento local utiliza Docker Compose para orquestrar todos os serviços da plataforma de pedidos. A arquitetura é composta por cinco serviços principais: API Gateway, Servico de Pedidos, Servico de Pagamentos, Servico de Estoque e banco de dados PostgreSQL. O arquivo docker-compose.yml define a configuracao completa, incluindo redes isoladas para comunicacao entre serviços, volumes persistentes para dados do banco, e variaveis de ambiente para configuracao de cada componente.

A configuracao de redes no Docker Compose é fundamental para isolar a comunicacao entre serviços. A rede frontend conecta o API Gateway aos serviços backend, enquanto a rede backend permite comunicacao entre os serviços de negocio e o banco de dados. Este isolamento simula a topologia de producao, onde diferentes camadas da aplicacao podem ter diferentes politicas de acesso. Os volumes persistentes garantem que dados do PostgreSQL sobrevivam a reinicializacoes dos containers, enquanto variaveis de ambiente permitem configuracao flexivel sem modificacao de codigo.

1.2 Estrutura de Rede e Volumes

A definicao de redes e volumes no docker-compose.yml segue boas praticas de isolamento e persistencia. A rede gateway-net é utilizada para comunicacao externa, permitindo que clientes acessem o API Gateway. As redes pedidos-net, pagamentos-net e estoque-net isolam a comunicacao entre cada servico e suas dependencias especificas. O volume postgres_data é montado no container do PostgreSQL, garantindo persistencia de dados entre reinicializacoes. Variaveis de ambiente como POSTGRES_DB, POSTGRES_USER e POSTGRES_PASSWORD sao definidas no arquivo .env, mantendo credenciais sensiveis fora do controle de versao.

2. Containerizacao e Versionamento

2.1 Dockerfiles Otimizados

Cada microsservico possui um Dockerfile otimizado seguindo praticas de seguranca e eficiencia. A estrutura utiliza multi-stage builds para separar o ambiente de build do ambiente de runtime, resultando em imagens finais significativamente menores. O primeiro estagio utiliza uma imagem completa do SDK para compilacao e execucao de testes, enquanto o estagio final copia apenas os artefatos necessarios para uma

imagem base minimalista. Este padrao reduz a superficie de ataque e o tempo de transferencia de imagens.

As boas praticas de seguranca implementadas incluem: execucao como usuario nao-root, reduzindo privilegios em caso de comprometimento; minimizacao de camadas através de instrucoes RUN concatenadas; utilizacao de imagens base oficiais e minimalistas como Alpine; definicao explicita de HEALTHCHECK para verificacao de integridade; e remocao de ferramentas de build e dependencias desnecessarias na imagem final. O versionamento de imagens segue o padrao semantico (major.minor.patch), com tags adicionais para ambientes (dev, staging, production) e identificadores de commit para rastreabilidade.

Boas Praticas em Dockerfiles

Pratica	Beneficio	Implementacao
Multi-stage builds	Reducao de tamanho	Separar build e runtime
Usuario nao-root	Seguranca	USER appuser no Dockerfile
.dockerignore	Contexto de build menor	Excluir arquivos nao necessarios
Camadas minimalistas	Cache eficiente	RUN apt-get && rm cache
Health checks	Confiabilidade	HEALTHCHECK CMD curl

3. Kubernetes - Producao Minima

3.1 Manifests Organizados

Os manifests do Kubernetes sao organizados em uma estrutura de diretorios que separa claramente os recursos por tipo e ambiente. A estrutura base/ contem recursos comuns a todos os ambientes, enquanto overlays/ contem variacoes especificas para dev, staging e production, seguindo o padrao Kustomize. Cada servico possui seus proprios manifests para Deployment, Service, ConfigMap e Secret, facilitando manutencao e permitindo deploy independente.

Os Deployments definem o estado desejado para cada microservico, incluindo numero de replicas, estrategia de atualizacao (rolling update com maxSurge e maxUnavailable), container image, ports, probes e resource requests/limits. Os Services expoe os Pods através de seletores, com ClusterIP para comunicacao interna e NodePort ou LoadBalancer para exposicao externa. ConfigMaps armazenam configuracoes nao sensiveis como URLs de servicos e timeouts, enquanto Secrets armazenam credenciais de banco de dados e chaves de API criptografadas em base64.

3.2 ConfigMaps, Secrets e Probes

O gerenciamento de configuracao segue o principio de separacao entre configuracao e codigo. ConfigMaps sao montados como arquivos ou variaveis de ambiente, permitindo alteracoes de configuracao sem reconstrucao de imagens. Secrets sao gerenciados através de solucoes externas como HashiCorp Vault ou AWS Secrets Manager, injetados no momento do deploy. A rotacao de secrets é automatizada através de scripts que atualizam os objetos Secret e reiniciam os Pods afetados.

Health probes sao essenciais para garantir disponibilidade. Liveness probes verificam se o container esta rodando corretamente, reiniciando-o em caso de falha. Readiness probes verificam se o container esta pronto para receber trafego, removendo-o do service endpoint ate que esteja saudavel. Startup probes permitem que containers com inicializacao lenta tenham tempo adicional antes que liveness probes comecem a verificar. A configuracao adequada destes probes é critica para evitar reinicializacoes em cascata e garantir rolling updates suaves.

4. Pipeline CI/CD

4.1 Estrutura do Pipeline

O pipeline de CI/CD é implementado utilizando GitHub Actions, com workflows definidos em arquivos YAML no diretorio `.github/workflows/`. O pipeline é acionado automaticamente em pushes para branches main e develop, e em pull requests. A estrutura é modular, com jobs sequenciais para build, test, security scan e deploy, cada um executando em um runner isolado com dependencias especificas.

O estagio de build compila cada microservico, executa testes unitarios e de integracao, e gera artefatos (binarios e imagens Docker). O estagio de testes executa suites de testes automatizados, incluindo testes end-to-end utilizando containers de serviços para simular dependencias. O estagio de seguranca executa scan de vulnerabilidades em dependencias (Dependabot, Snyk) e analise estatica de codigo (SonarQube). O estagio de deploy utiliza ambientes do GitHub Actions para promover artefatos através de dev, staging e production, com aprovacoes manuais para ambientes criticos.

4.2 Gestao de Secrets e Validacoes

Secrets do pipeline sao armazenados de forma segura utilizando GitHub Secrets, criptografados em repouso e injetados no ambiente de execucao apenas quando necessario. Credenciais de registry, chaves de API e tokens de deploy sao armazenados com escopo limitado a repositorios especificos. A rotacao de secrets é realizada através de procedimentos documentados, com alertas configurados para expiracao de credenciais.

Validacoes automatizadas incluem lint de codigo (ESLint, Pylint), formatacao (Prettier, Black), testes unitarios com cobertura minima de 80%, testes de integracao, scan de vulnerabilidades em imagens (Trivy), e validacao de manifests Kubernetes (kubeval, kubeconform). Estas validacoes atuam como gates que impedem progressao de codigo que nao atenda aos padroes de qualidade estabelecidos, garantindo que

apenas código validado alcance ambientes de produção.

5. Observabilidade, Deploy e Escala

5.1 Métricas, Logs e Tracing

A stack de observabilidade é baseada em Prometheus para coleta de métricas, Grafana para visualização e alertas, Loki para agregação de logs, e Jaeger para tracing distribuído. Cada microsserviço expõe métricas no formato Prometheus através de bibliotecas de instrumentação, incluindo contadores de requisições, histograms de latência, e gauges de recursos. Dashboards pre-configurados em Grafana fornecem visibilidade sobre saúde do sistema, performance e tendências.

Logs são estruturados em formato JSON e enviados para stdout/stderr, coletados pelo runtime de containers e encaminhados para Loki. A correlação entre logs de diferentes serviços é facilitada por trace IDs propagados através de headers HTTP. O tracing distribuído é implementado com OpenTelemetry, instrumentando automaticamente bibliotecas HTTP e de mensageria. Jaeger visualiza o fluxo de requisições através de serviços, identificando latências e erros em cada etapa do processamento.

5.2 Estratégia de Deploy

A estratégia de deploy escolhida é rolling update, configurada nos Deployments do Kubernetes com maxSurge de 25% e maxUnavailable de 25%. Esta configuração permite que novas réplicas sejam criadas progressivamente enquanto réplicas antigas são terminadas, garantindo disponibilidade contínua durante atualizações. Em casos de problemas detectados, rollback é executado automaticamente quando probes falham, ou manualmente através de comandos kubectl rollout undo.

Para releases de maior risco, como mudanças de schema de banco de dados ou refatorações significativas, o padrão blue-green pode ser implementado utilizando dois Deployments paralelos e um Service que alterna entre eles. Esta abordagem permite validação completa da nova versão em ambiente de produção antes de direcionar tráfego, com rollback instantâneo em caso de problemas. Canary releases podem ser implementados através de múltiplos Services com proporções de tráfego controladas por Ingress ou Service Mesh.

5.3 Estratégia de Escalabilidade

A escalabilidade é implementada em múltiplas dimensões. No nível de Pods, Horizontal Pod Autoscaler (HPA) escala réplicas com base em métricas de CPU e memória, com thresholds configurados em 70% de utilização para evitar thrashing. Para métricas customizadas como comprimentos de fila ou latência de requisições, o Kubernetes Metrics Server é estendido com adaptadores para Prometheus. Vertical Pod Autoscaler (VPA) analisa historicamente o consumo de recursos e recomenda valores ideais de requests/limits.

No nível de nodes, o Cluster Autoscaler monitora recursos não alocados e Pods pendentes, adicionando nodes quando necessário e removendo nodes subutilizados. Esta escalabilidade automática é fundamental para lidar com picos de tráfego como campanhas promocionais, onde o número de réplicas pode aumentar de 3 para 20 ou mais em questão de minutos. A configuração adequada de Pod Disruption Budgets (PDB) garante que o número mínimo de réplicas seja mantido durante operações de manutenção do cluster.

6. Infraestrutura como Código (IaC)

A infraestrutura é definida como código utilizando Terraform, permitindo provisionamento reproduzível e versionado de recursos de cloud. A estrutura de módulos separa recursos por componente: rede (VPC, subnets, security groups), cluster Kubernetes (EKS, node groups), serviços gerenciados (RDS, ElastiCache), e monitoramento (CloudWatch, Prometheus). O state do Terraform é armazenado remotamente em S3 com locking via DynamoDB, permitindo colaboração segura entre membros da equipe.

A escolha do Terraform justifica-se pela capacidade de gerenciar recursos multi-cloud, infraestrutura declarativa idempotente, e amplo ecossistema de providers. Módulos reutilizáveis encapsulam padrões comuns, como cluster Kubernetes com logging e monitoring habilitados. Workspaces permitem gerenciar múltiplos ambientes (dev, staging, production) com a mesma configuração, variando apenas parâmetros específicos. O pipeline de infraestrutura aplica mudanças automaticamente em ambientes de desenvolvimento, com aprovação manual para staging e produção, garantindo que mudanças de infraestrutura sigam o mesmo fluxo de governança que mudanças de aplicação.

VIDEO DE APRESENTACAO

Assista ao vídeo pitch de apresentação deste trabalho:

<https://youtu.be/VXn-jSdxflY>